

01 - Project Overview

Hardware ERP, Module 1: Authentication and User Management. Read this first.

What is this?

Hardware ERP is software for **one hardware shop** - the kind that sells locks, hinges, screws, handles and door closers. It will eventually handle billing, stock, purchases and accounts.

This document set covers **Module 1 only**: the part that answers two questions before anything else can happen.

Question	Name	Example
Who are you?	Authentication	Karthik types his mobile number and password
What may you do?	Authorization	Karthik can raise a bill; he cannot see purchase cost

WHY

Why build this first

Every other module needs to know who is asking. A bill has to record who raised it. A stock adjustment has to record who made it. If you build billing first and add login later, you end up with a pile of records that have no owner and no way to prove who did what.

What Module 1 actually does

- A person signs in with **either** their mobile number **or** their email
- The system gives them a short-lived digital pass called a token
- The pass is checked on every single request
- The shop owner creates staff accounts. **Nobody can sign themselves up**
- Each account has one role. Each role is a bundle of permissions
- Five wrong passwords lock the account for 15 minutes
- A forgotten password is reset through a single-use email link
- Every security event is written to a log that is never edited

The words you need

Technical word, then what it actually means, then why we use it.

Backend

The program that holds the rules and talks to the database. It runs on a server. Nobody sees it directly.

Like: the shop's back office, where the ledgers are kept and the rules are enforced. Customers never walk in there.

Ours is written in **Java** using **Spring Boot**.

Frontend

The screens a person looks at and clicks. It runs inside the web browser.

Like: the shop counter. It is where people interact, but it holds no authority of its own. It just passes requests to the back office.

Ours is written in **React** with **TypeScript**.

Database

The place where information is stored permanently, so it survives the program being switched off.

Like: the steel almirah full of ledgers. Close the shop, come back tomorrow, everything is still there.

Ours is **PostgreSQL**.

API

The agreed list of requests the frontend may make of the backend, and what each one gives back.

Like: the order slip format between the counter and the godown. The slip has fixed boxes. Fill them in correctly and you get your goods.

JWT (JSON Web Token)

A short-lived digital ID card that the backend hands out after a correct sign-in. The browser attaches it to every later request.

Like: the visitor badge at a factory gate. The guard checks it each time you pass a door. It expires, and it says nothing about your salary - only who you are.

IMPORTANT

A JWT is signed, not secret

Anyone holding the token can read what is inside it. It cannot be forged without the server's key, but it can be read. That is exactly why our tokens carry only a user id and a version number - no name, no role, no permission list.

Refresh token

A longer-lived key, used to get a fresh JWT when the old one expires, without making the person type their password again.

Like: the gate pass renewal slip. Hand in the old one, get a fresh badge.

Roles and permissions

A **permission** is one specific thing you may do, such as `USER_MANAGE` or `PRODUCT_VIEW_COST`.

A **role** is a named bundle of permissions, such as `STAFF` or `MANAGER`.

Like: a job title. "Counter staff" is the title. "May issue bills, may not see purchase cost" is what the title actually permits.

The four roles that ship with the system:

Role	Who they are	Can see purchase cost?
OWNER	The shop owner	Yes - everything
MANAGER	Day-to-day manager	Yes
ACCOUNTANT	Books and payments	Yes
STAFF	Billing counter	No

WHY

Why STAFF cannot see cost

If counter staff know you bought a lock for Rs 180, they know your margin on the Rs 260 sale. That is the owner's business, not the counter's. This rule is enforced by the backend, not by hiding a number on the screen. A hidden number can still be read by anyone who opens the browser's developer tools.

Flyway

A tool that keeps a numbered list of every change ever made to the database structure, and applies any that have not run yet.

Like: a maintenance logbook for the almirah. "Change 1: added a shelf.

Change 2: added a lock." Anyone can rebuild it exactly by following the log.

Swagger

A web page, generated automatically from the backend code, that lists every API and lets you try it in the browser.

Like: the printed menu at a restaurant, except you can order straight off the page.

Postman

A desktop app for sending API requests and checking the answers, with no frontend involved at all.

Like: phoning the godown directly instead of going through the counter.

Useful when you want to know whether the godown itself is answering correctly.

Tests

Small programs that call our code and check it did the right thing. They run automatically and fail loudly.

Like: the stock audit. You are not hunting a specific theft; you are checking that the numbers still add up after every change.

Module 1 has **149 tests**.

How one request travels

When Karthik clicks "Sign in", this is the journey:



Every layer exists because it does one job. If the password check lived in the screen, anyone could skip it by not using our screen.

The three pieces, running together



Three programs. You start all three on your own laptop.

IMPORTANT

One shop, one system

This is deliberately **not** a multi-company cloud product. One Spring Boot application, one React application, one PostgreSQL database. Simpler to build, simpler to fix, and far simpler to back up.

What exists in Module 1 today

Area	Amount
Backend source files	88 Java files
Backend tests	11 files, 149 test methods
Database migrations	2 - schema, and development seed data
API endpoints	25
Frontend	71 files, 9 pages
Postman requests	54, covering all 25 endpoints

VERIFY

What has actually been proven, and what has not

The frontend production build has been run and passes. The backend static consistency check passes. The backend has **not** been compiled and the tests have **not** been executed, because the authoring environment could not reach Maven Central or run Docker. Document 16 lists the exact status of every gate. Do not treat "the code exists" as "the code works" until you have run `mvn clean verify` yourself.

Where to go next

If you want to	Read
Install the tools on your laptop	02 - Development Environment Setup
Install and understand PostgreSQL	03 - PostgreSQL Setup
Understand database migrations	04 - Flyway Database Migration
Understand the backend layout	05 - Backend Architecture

If you want to	Read
Test the APIs in a few minutes	09 - Postman API Testing
Understand login and JWT deeply	07 - Authentication, JWT and Security
Check what is genuinely finished	16 - Module 1 Complete Verification
Fix an error you have hit	17 - Troubleshooting Guide